

Microservicio GraphQL para Gestión de Películas - Sakila 1.0.0



Nombre: Alejandro Miguel Pardo Romano

Docente: Miguel Ángel Pacheco

Materia: Tecnologías Web II

Gestión: 1-2024

Descripción

Este proyecto es un microservicio creado con **GraphQL** que se conecta con la API REST del sistema **Sakila** para obtener información de películas. Se encarga de:

- Autenticarse automáticamente con la API Sakila.
- Consultar la lista de películas desde la API REST.
- Exponer un endpoint **GraphQL** para que se pueda consultar esta información de manera más flexible.

Tecnologías Utilizadas

- **Node.js** para el backend.
- **Apollo Server** para el servidor GraphQL.
- **Axios** para hacer peticiones HTTP.
- **MySQL** para la base de datos.
- **dotenv** para manejar variables de entorno.

Instalación

1. Clonar el Repositorio

bash

Copiar

- `git clone https://github.com/tuusuario/microservicio-films-graphql.git`

2. Instalar Dependencias

Ve al directorio del proyecto y ejecuta:

bash

Copiar

- `cd microservicio-films-graphql`
- `npm install`

3. Configurar Variables de Entorno

Crea un archivo `.env` en la raíz del proyecto con este contenido:

env

Copiar

- `API_URL=http://localhost:3000`
- `USERNAME=admin`
- `PASSWORD=admin123`

4. Iniciar el Servidor

Para arrancar el servidor, usa:

bash

Copiar

- `npm start`

Ahora podrás acceder a **GraphQL Playground** en <http://localhost:4000/>.

Ejemplo de Consulta

Consulta de Películas

graphql

Copiar

- `query {`
- `films {`
- `film_id`
- `title`
- `description`
- `release_year`
- `}`
- `}`

Respuesta Esperada

json

```
Response 200 | 135ms | 169.9KB

{
  "data": {
    "films": [
      {
        "film_id": "1",
        "title": "ACADEMY DINOSAUR",
        "description": "A Epic Drama of a Feminist And a Mad Scientist who must Battle a Teacher in The Canadian Rockies",
        "release_year": 2006
      },
      {
        "film_id": "2",
        "title": "ACE GOLDFINGER",
        "description": "A Astounding Epistle of a Database Administrator And a Explorer who must Find a Car in Ancient China",
        "release_year": 2006
      },
      {
        "film_id": "3",
        "title": "ADAPTATION HOLES",
        "description": "A Astounding Reflection of a Lumberjack And a Car who must Sink a Lumberjack in A Baloon Factory",
        "release_year": 2006
      },
      {
        "film_id": "4",
        "title": "AFFAIR PREJUDICE",
        "description": "A Fanciful Documentary of a Frisbee And a Lumberjack who must Chase a Monkey in A Shark Tank",
        "release_year": 2006
      },
      {
        "film_id": "5",
        "title": "AFRICAN EGG",
        "description": "A Fast-Paced Documentary of a Pastry Chef And a Dentist who must Pursue a Forensic Psychologist in The Gulf of Mexico",
        "release_year": 2006
      },
      {

```

Pruebas

1. Consulta con Token Válido

Descripción:

Realiza la consulta de películas con el **token JWT**.

Pasos:

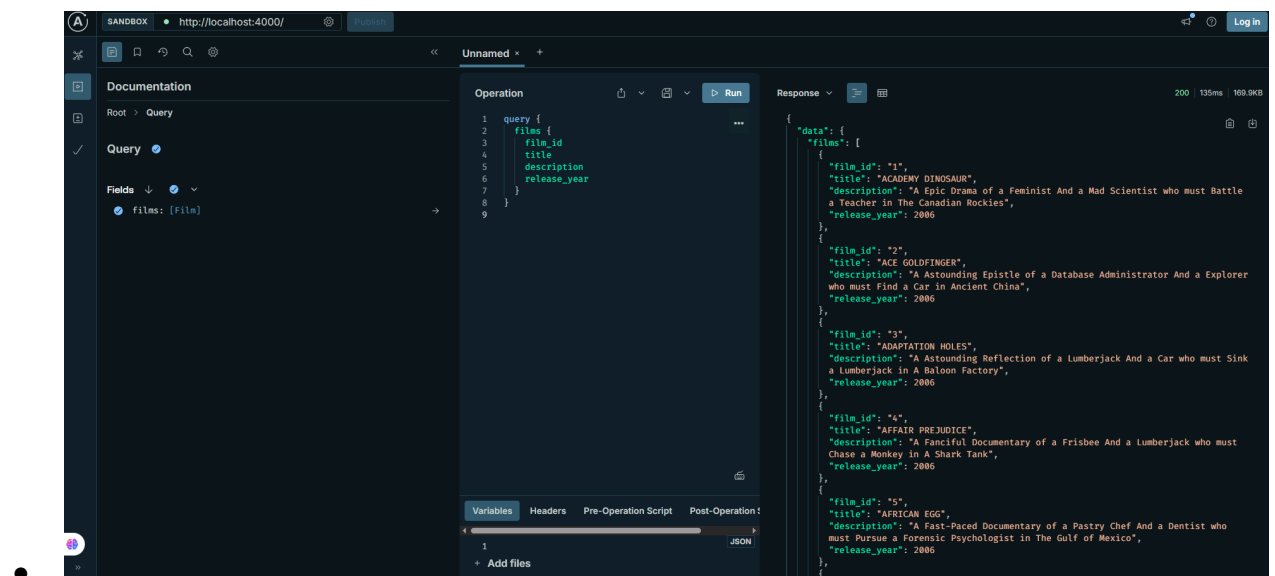
1. Abre **GraphQL Playground** en <http://localhost:4000/>.
2. Agrega el **token JWT** en los headers:

json

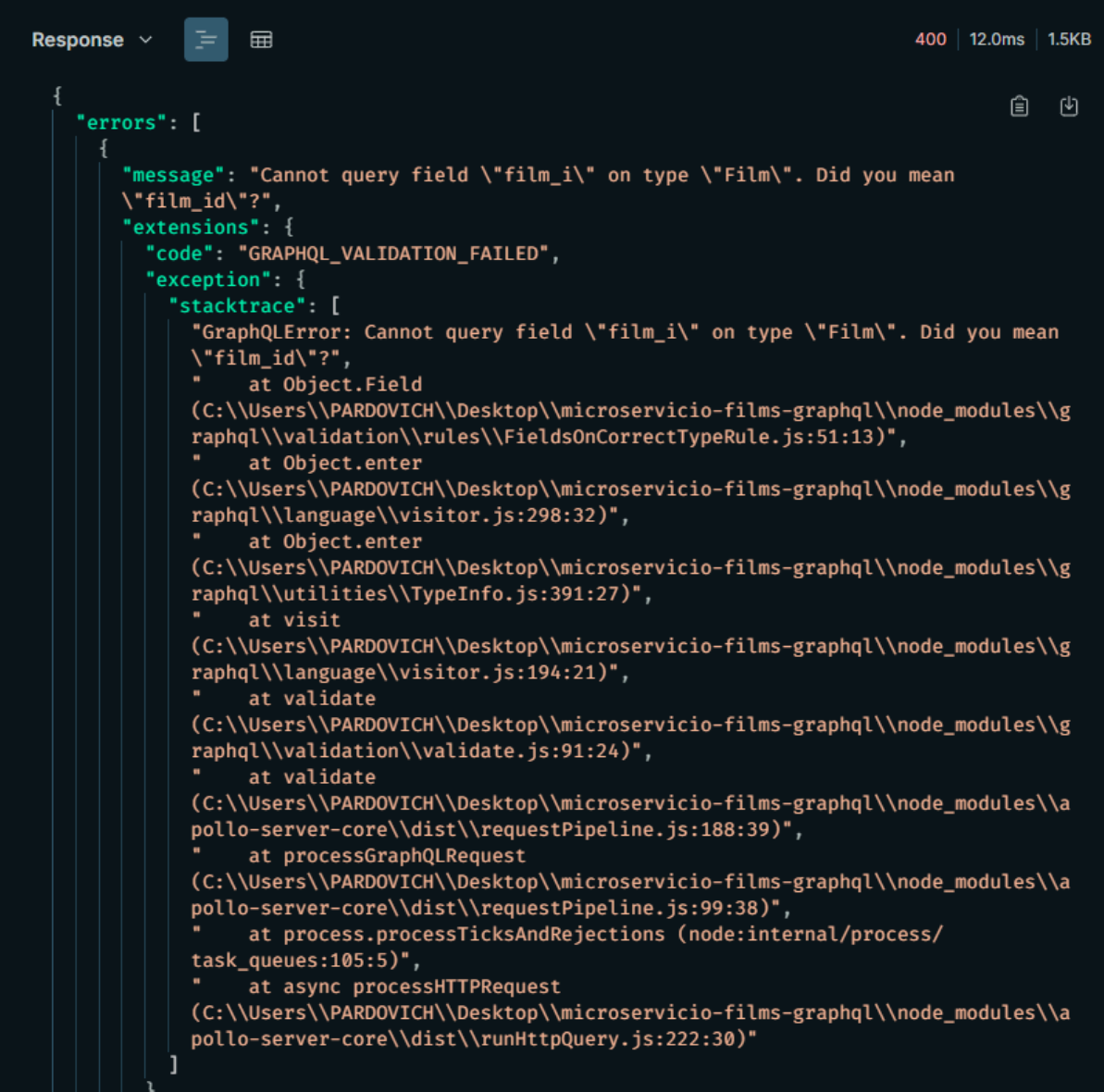
Copiar

- ```
{
```
- ```
  "Authorization": "Bearer <tu_token_jwt>"
```
- ```
}
```

### Captura de Pantalla:



## 2. Consulta sin Token (Debería Fallar)



The screenshot shows the 'Response' tab in GraphQL Playground. The response is a JSON object with an 'errors' array containing one error object. The error message is 'Cannot query field \"film\_i\" on type \"Film\". Did you mean \"film\_id\"?'. The error code is 'GRAPHQL\_VALIDATION\_FAILED'. The stack trace shows the error originated in the GraphQL validation rules, specifically in the 'FieldsOnCorrectTypeRule.js' file.

```
{
 "errors": [
 {
 "message": "Cannot query field \"film_i\" on type \"Film\". Did you mean \"film_id\"?",
 "extensions": {
 "code": "GRAPHQL_VALIDATION_FAILED",
 "exception": {
 "stacktrace": [
 "GraphQLError: Cannot query field \"film_i\" on type \"Film\". Did you mean \"film_id\"?",
 " at Object.Field",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\graphql\\validation\\rules\\FieldsOnCorrectTypeRule.js:51:13)",
 " at Object.enter",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\graphql\\language\\visitor.js:298:32)",
 " at Object.enter",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\graphql\\utilities\\TypeInfo.js:391:27)",
 " at visit",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\graphql\\language\\visitor.js:194:21)",
 " at validate",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\graphql\\validation\\validate.js:91:24)",
 " at validate",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\apollo-server-core\\dist\\requestPipeline.js:188:39)",
 " at processGraphQLRequest",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\apollo-server-core\\dist\\requestPipeline.js:99:38)",
 " at process.processTicksAndRejections (node:internal/process/task_queues:105:5)",
 " at async processHttpRequest",
 "(C:\\Users\\PARDOVICH\\Desktop\\microservicio-films-graphql\\node_modules\\apollo-server-core\\dist\\runHttpQuery.js:222:30)"
]
 }
 }
 }
]
}
```

### Descripción:

Intenta hacer la misma consulta sin enviar el token y debería devolver un error de autenticación.

### Pasos:

1. Elimina el header de **Authorization** en **GraphQL Playground**.
2. Realiza la consulta.

# Estructura del Proyecto

El proyecto tiene la siguiente estructura de carpetas:

- /microservicio-films-graphql
  - |— src/
    - | |— apiClient.js # Lógica para consumir la API REST externa
    - | |— resolvers.js # Resolvers de GraphQL
    - | |— schema.graphql # Esquema GraphQL
    - | |— server.js # Configuración y arranque del servidor
  - |— .env # Variables de entorno
  - |— package.json # Dependencias y scripts del proyecto
  - |— README.md # Este archivo

## Capturas de Pantalla

### 1. Consulta de Películas con Token Válido

